ARQUITECTURA DE MICROSERVICIOS

Service Discovery con Eureka y Circuit Breaker con Resilience4j

Taller sobre Patrones de Arquitectura de Microservicios

Pablo Andrés Guzmán Alarcón · A00399523

Stick Martinez · A00398704

Alejandro Troya · A00399865

Código fuente: https://github.com/pabloandressoftware/FutureXCourseCatalog/tree/feature/resilience4j

¿Qué problema resolvemos?

Contexto

1 · Encontrar al otro servicio

- Las URL fijas (`localhost:8080`) se rompen al cambiar de puerto o de máquina.
- Hay varias instancias y aparecen o desaparecen.

→ **Service Discovery** (Eureka)

2 · Sobrevivir cuando falla

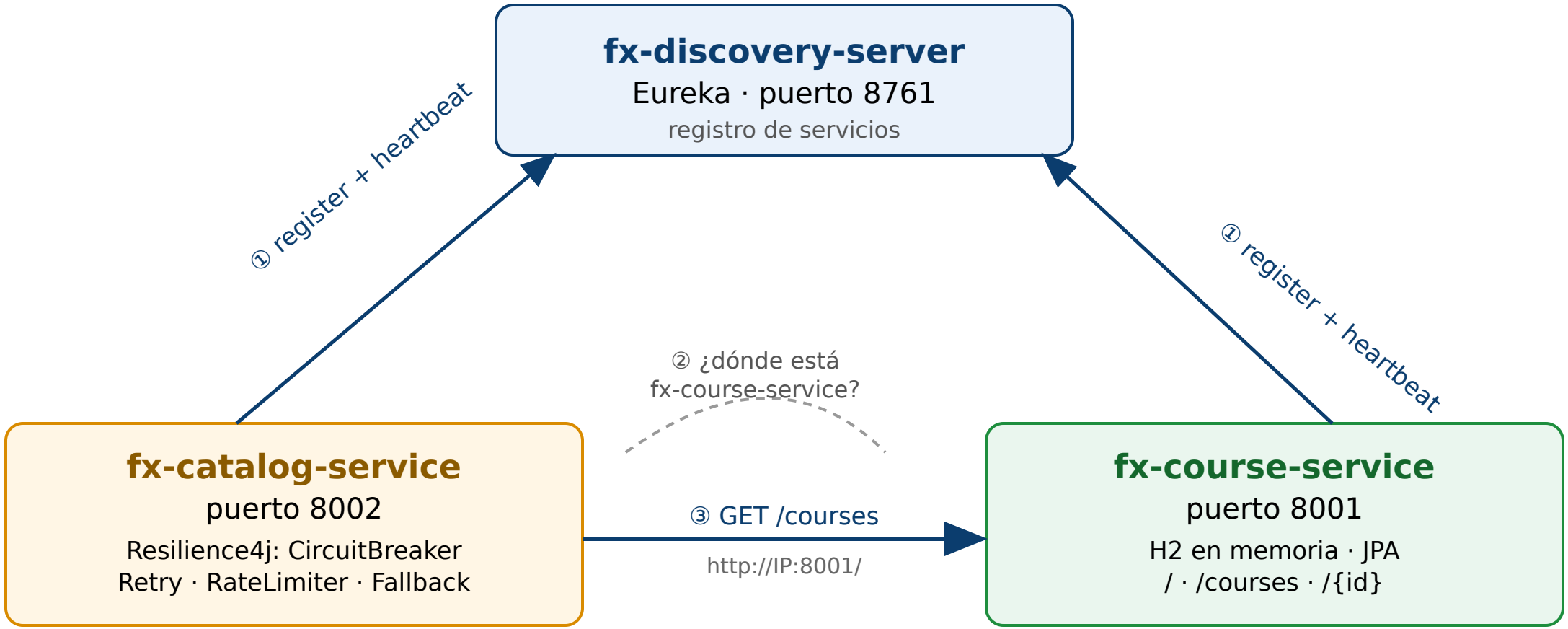
- Un servicio caído o lento arrastra a quien lo llama (**fallo en cascada**).
- El cliente recibe errores 500.

→ **Circuit Breaker** (Resilience4j)

Caso del taller: `fx-catalog-service` necesita datos de `fx-course-service`.

Arquitectura del taller

Tres servicios Spring Boot 3.3.3 · Spring Cloud 2023.0.3 · Java 17+



Parte 1 · Cómo funciona el Service Discovery

40 % · Comprensión del Service Discovery

① **Registro:** al arrancar, cada servicio se registra en Eureka con su nombre (`spring.application.name`) y envía un *heartbeat* cada 30 s.

② **Búsqueda:** el catálogo pregunta por el nombre lógico `fx-course-service`, no por una IP.

③ **Llamada:** usa la URL que Eureka devolvió.

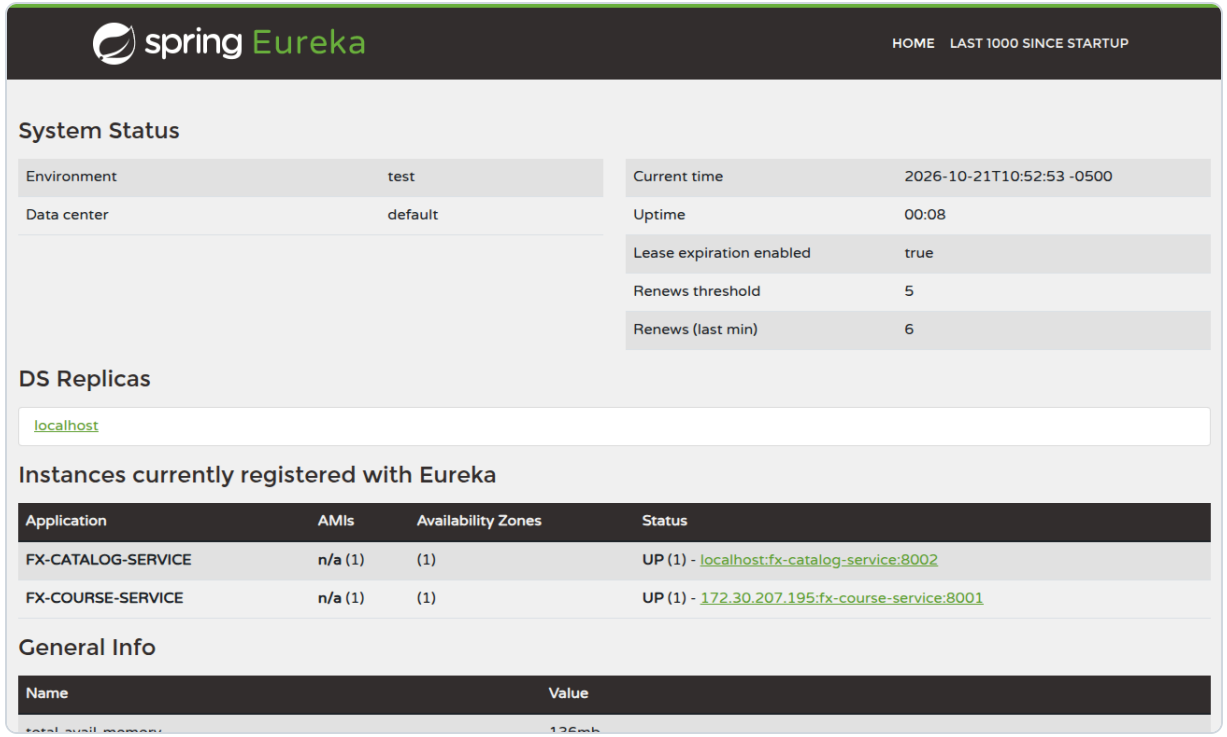
```
InstanceInfo i = client.getNextServerFromEureka(
        "fx-course-service", false);   // ② descubrir
String url = i.getHomePageUrl();       //
http://IP:8001/

restTemplate.getForObject(
        url + "courses", String.class); // ③ llamar
```

```
# fx-course-service
spring.application.name=fx-course-service
server.port=8001
eureka.client.service-url.defaultZone=
        http://localhost:8761/eureka/
```

Demostración · Eureka y los endpoints

Los tres endpoints funcionan gracias al descubrimiento



Panel Eureka `:8761` — ambos servicios en **UP**.

Endpoint (:8002)	Respuesta
`/`	Welcome to FutureX Course Catalog Course App Home
`/catalog`	3 cursos (JSON) desde fx-course-service
`/firstcourse`	Spring Boot Fundamentals

> **Hallazgo:** al reiniciar el servicio de cursos, el catálogo falló unos segundos (`No matches for the virtual host name`): Eureka tiene **consistencia eventual** (cachés ~30 s). Ajuste: `registry-fetch-interval-seconds: 10` .

El problema: sin protección, todo falla

Línea base · Antes

Se detiene `fx-course-service` y se llama al catálogo **original**:

```
GET /catalog   (sin Resilience4j, course-service caído)
{"status":500,"error":"Internal Server Error","path":"/catalog"}  [HTTP 500]
GET /firstcourse
{"status":500,"error":"Internal Server Error","path":"/firstcourse"} [HTTP 500]
```

- El error se **propaga** al cliente.
- Cada petición sigue intentando contactar al servicio caído (espera, hilos ocupados).
- No hay recuperación controlada.

Parte 2 · Qué se implementó con Resilience4j

60 % · Implementación de Resilience4j

pom.xml

- `spring-cloud-starter-circuitbreaker-resilience4j`
- `spring-boot-starter-aop`
- `spring-boot-starter-actuator`

application.yml

- CircuitBreaker `courseService`
- Retry (3 intentos · 500 ms)
- RateLimiter (5 / 10 s)
- Endpoints de Actuator

Código Java

- `@CircuitBreaker`, `@Retry`, `@RateLimiter`
- 3 métodos `fallback*`
- `RestTemplate` con timeouts

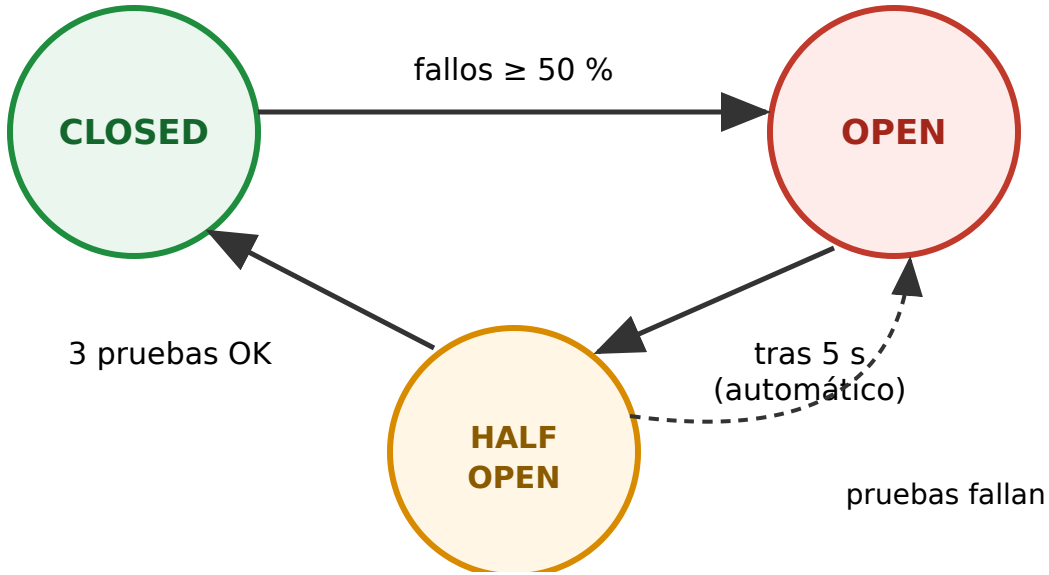
Endpoint	Anotaciones	Fallback
`/`	`@CircuitBreaker`	`fallbackGetCatalogHome`
`/catalog`	`@Retry` + `@CircuitBreaker`	`fallbackGetCatalog`
`/firstcourse`	`@CircuitBreaker` + `@RateLimiter`	`fallbackGetSpecificCourse`

Configuración del Circuit Breaker

Propiedades del taller en application.yml

```
resilience4j.circuitbreaker.instances.courseService:
  registerHealthIndicator: true
  slidingWindowSize: 10
  minimumNumberOfCalls: 5
  failureRateThreshold: 50
  waitDurationInOpenState: 5s
  automaticTransitionFromOpenToHalfOpenEnabled: true
  permittedNumberOfCallsInHalfOpenState: 3
  eventConsumerBufferSize: 10
```

- Ventana de **10** llamadas; evalúa desde la 5.ª.
- ≥ **50 %** de fallos → abre el circuito.
- Abierto **5 s**, luego prueba con **3** llamadas.



Anotaciones y métodos de fallback

CatalogController · /catalog

```
@GetMapping("/catalog")
    @Retry(name = "courseService", fallbackMethod =
"fallbackGetCatalog")
    @CircuitBreaker(name = "courseService")
    public String getCatalog() {
        String courses =
restTemplate.getForObject(courseServiceUrl() + "courses",
String.class);
        return "Our courses are " + courses;
    }
```

```
public String fallbackGetCatalog(Exception ex) {
        log.warn("fallbackGetCatalog: {}", ex.toString());
        return "Fallback: Unable to retrieve course catalog (" +
reason(ex) + ").";
    }
```

Reglas del fallback

- Mismo tipo de retorno y mismos parámetros + un `Exception` final.
- Se ejecuta si la llamada **falla** o si el circuito está **abierto**.
- `reason()` distingue la causa: servicio caído, circuito OPEN o límite de tasa.

Decisiones de diseño (lo que aprendí al probar)

Hallazgos durante la demostración

① El fallback va en `@Retry` (la más externa) en `/catalog`

Orden de aspectos: `Retry ( CircuitBreaker ( RateLimiter ( … ) ) )`. Un fallback en `@CircuitBreaker` absorbe el error y el Retry **nunca reintenta**.

② `@Retry` ignora `CallNotPermittedException`

En la 1.ª demo, con el circuito OPEN el Retry seguía reintentando (500 ms × 2 por llamada) y anulaba el *fail fast*. Con `retry.ignoreExceptions` la respuesta es inmediata.

③ Timeouts en el `RestTemplate` (1 s conexión · 2 s lectura)

Una llamada colgada debe fallar rápido para que el circuito la cuente como fallo.

Demo (1/2) · Servicio caído → fallback → circuito OPEN

Salida real de scripts/demo.sh

Escenario 2: se detiene `fx-course-service`.

```
  [llamada 1]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (course service unavailable).
  [llamada 2]
  GET /catalog -> [HTTP 200] Fallback: Unable to retrieve course catalog (circuit breaker is OPEN).
  Circuit Breaker 'courseService': OPEN | failureRate 50.0% | failedCalls 4 | notPermitted 5
```

Escenario 3: con el circuito OPEN se rechaza *sin intentar la llamada*.

```
  GET / -> [HTTP 200] Fallback: Welcome to FutureX Course Catalog, but the course service is not available
(circuit breaker is OPEN).
  GET /firstcourse -> [HTTP 200] Fallback: Unable to retrieve the first course (circuit breaker is OPEN).
  Circuit Breaker 'courseService': OPEN | failureRate 50.0% | failedCalls 4 | notPermitted 7
```

HTTP 200 con mensaje controlado en lugar de **HTTP 500**.

Demo (2/2) · Recuperación, Retry y RateLimiter

Salida real de scripts/demo.sh

Escenario 4: vuelve el servicio → `OPEN → HALF_OPEN → CLOSED` sin intervención.

```
  GET / -> [HTTP 200] Welcome to FutureX Course Catalog Course App Home
  GET /catalog -> [HTTP 200] Our courses are [{"courseid":1,"coursename":"Spring Boot
Fundamentals","author":"John Doe"},{"courseid":2,"coursename":"Advanced Java Progra
  GET /firstcourse -> [HTTP 200] Our first course is Spring Boot Fundamentals
  Circuit Breaker 'courseService': CLOSED | failureRate -1.0% | failedCalls 0 | notPermitted 0
```

Escenario 5: RateLimiter (5 / 10 s)

```
  GET /firstcourse -> [HTTP 200] Our first course is
Spring Boot Fundamentals
  GET /firstcourse -> [HTTP 200] Fallback: Unable to
retrieve the first course (rate limit exceeded).
  GET /firstcourse -> [HTTP 200] Fallback: Unable to
retrieve the first course (rate limit exceeded).
```

Retry (`/actuator/retryevents`)

```
   11:00:51.110 RETRY intento 1
   11:00:51.614 RETRY intento 2
   11:00:52.118 ERROR intento 3
   11:00:52.166 RETRY intento 1
```

Cómo Resilience4j mejora la resiliencia

Evita los fallos en cascada

Patrón	Qué evita
Circuit Breaker	Seguir llamando a un servicio que está fallando; le da tiempo a recuperarse y no gasta recursos.
Fallback	Error 500 al usuario → respuesta degradada pero controlada.
Retry	Fallos transitorios (red, reinicio, caché de Eureka).
RateLimiter	Ráfagas de tráfico que saturan al servicio.
Bulkhead · TimeLimiter	Aislar recursos por dependencia y acotar la duración (no usados aquí).

Resultado medido: el catálogo **sigue disponible**, protege al servicio caído mientras está OPEN y **se recupera solo**.

Hystrix vs Resilience4j

Cómo mejoraba Hystrix la resiliencia y por qué se reemplaza

Hystrix (Netflix) envolvía cada llamada en un `HystrixCommand`: circuit breaker sobre ventana de tiempo (10 s), *thread pools* por dependencia (bulkhead), timeouts, fallback y dashboard (Turbine).

Aspecto	Hystrix	Resilience4j
Estado	Mantenimiento desde 2018; eliminado de Spring Cloud 2020.0	**Activo; soportado por Spring Boot 3**
Diseño	Objeto `HystrixCommand`; deps. de Netflix	Decoradores funcionales, modular y ligero
Aislamiento	Thread pools (más sobrecarga)	Semáforos; thread pool opcional
Ventana	Por tiempo	Por conteo o por tiempo
Patrones	CB, fallback, aislamiento	CB, Retry, RateLimiter, Bulkhead, TimeLimiter, Cache
Métricas	Hystrix Dashboard	Micrometer + Actuator

Conclusiones

Cierre

- **Service Discovery:** los servicios se encuentran por *nombre lógico*; Eureka evita URL fijas.
- **Resilience4j:** Circuit Breaker + Fallback + Retry + RateLimiter mantienen el sistema **disponible y degradado** en vez de caído.
- **Lo importante al probar:** orden de los aspectos, `ignoreExceptions` y la caché de Eureka cambian el comportamiento real.
- Monitoreo con Actuator: `/actuator/health`, `circuitbreakers`, `circuitbreakerevents`.

Código: github.com/pabloandressoftware/FutureXCourseCatalog · rama `feature/resilience4j`

¿Preguntas?